

ASSIGNMENT-11.2

2303a510I9

BATCH:30

TASK 1:

Stack Using AI Guidance

PROMPT:

design and implement a Stack data structure supporting basic stack operations.

CODE & OUTPUT:

```
Ai Ass 11.2.py X
Ai Ass 11.2.py > Stack > size
1 #design and implement a Stack data structure supporting basic stack operations.
2 class Stack:
3     def __init__(self):
4         self.items = []
5     def push(self, item):
6         """Add an item to the top of the stack."""
7         self.items.append(item)
8     def pop(self):
9         """Remove and return the item at the top of the stack. Raises IndexError if stack is empty."""
10        if self.is_empty():
11            raise IndexError("pop from empty stack")
12        return self.items.pop()
13    def peek(self):
14        """Return the item at the top of the stack without removing it. Raises IndexError if stack is empty."""
15        if self.is_empty():
16            raise IndexError("peek from empty stack")
17        return self.items[-1]
18    def is_empty(self):
19        """Return True if the stack is empty, False otherwise."""
20        return len(self.items) == 0
21    def size(self):
22        """Return the number of items in the stack."""
23        return len(self.items)
24
25 # Example usage:
26 if __name__ == "__main__":
27     stack = Stack()
28     stack.push(1)
29     stack.push(2)
30     stack.push(3)
31     print("Top item:", stack.peek()) # Output: Top item: 3
32     print("Stack size:", stack.size()) # Output: Stack size: 3
33     print("Popped item:", stack.pop()) # Output: Popped item: 3
34     print("Stack is empty?", stack.is_empty()) # Output: Stack is empty? False
35     print("Popped item:", stack.pop()) # Output: Popped item: 2
36     print("Popped item:", stack.pop()) # Output: Popped item: 1
37     print("Stack is empty?", stack.is_empty()) # Output: Stack is empty? True
38     # stack.pop() # Uncommenting this line will raise IndexError: pop from empty
```

```
PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "C:/Users/Ramya Sri/OneDrive/New folder/AI Assistant coding/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\.venv\Scripts\python.exe' 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\main.py'
Top item: 3
Stack size: 3
Popped item: 3
Stack is empty? False
Popped item: 2
Popped item: 1
Stack is empty? True
(.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "C:/Users/Ramya Sri/OneDrive\New folder\AI Assistant coding\.venv\Scripts/Activate.ps1"
```

TASK 2: Queue Design

PROMPT:

create a Queue data structure following FIFO principles

CODE & OUTPUT:

```

39 #create a Queue data structure following FIFO principles
40 class Queue:
41     def __init__(self):
42         self.items = []
43     def enqueue(self, item):
44         """Add an item to the end of the queue."""
45         self.items.append(item)
46     def dequeue(self):
47         """Remove and return the item at the front of the queue. Raises IndexError if queue is empty."""
48         if self.is_empty():
49             raise IndexError("dequeue from empty queue")
50         return self.items.pop(0)
51     def peek(self):
52         """Return the item at the front of the queue without removing it. Raises IndexError if queue is empty."""
53         if self.is_empty():
54             raise IndexError("peek from empty queue")
55         return self.items[0]
56     def is_empty(self):
57         """Return True if the queue is empty, False otherwise."""
58         return len(self.items) == 0
59     def size(self):
60         """Return the number of items in the queue."""
61         return len(self.items)
62 # Example usage:
63 if __name__ == "__main__":
64     queue = Queue()
65     queue.enqueue(1)
66     queue.enqueue(2)
67     queue.enqueue(3)
68     print("Front item:", queue.peek()) # Output: Front item: 1
69     print("Queue size:", queue.size()) # Output: Queue size: 3
70     print("Dequeued item:", queue.dequeue()) # Output: Dequeued item: 1
71     print("Queue is empty?", queue.is_empty()) # Output: Queue is empty? False
72     print("Dequeued item:", queue.dequeue()) # Output: Dequeued item: 2
73     print("Dequeued item:", queue.dequeue()) # Output: Dequeued item: 3
74     print("Queue is empty?", queue.is_empty()) # Output: Queue is empty? True
75     # queue.dequeue() # Uncommenting this line will raise IndexError: dequeue from empty
76

```

```
PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "C:/Users/Ramya Sri/OneDrive/New folder/AI Assistant coding/.venv/Scripts/.venv" PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\.venv\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61622' '--' 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\'
Front item: 1
Queue size: 3
Dequeued item: 1
Queue is empty? False
Dequeued item: 2
Queue is empty? False
Dequeued item: 2
Dequeued item: 2
Dequeued item: 3
Queue is empty? True
Queue is empty? True
(.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\.venv
```

TASK 3:

Singly Linked List Construction

PROMPT:

build a singly linked list supporting insertion and traversal.

CODE & OUTPUT:

```
79 #build a singly linked list supporting insertion and traversal.
80 class Node:
81     def __init__(self, data):
82         self.data = data
83         self.next = None
84 class SinglyLinkedList:
85     def __init__(self):
86         self.head = None
87     def insert(self, data):
88         """Insert a new node with the given data at the end of the list."""
89         new_node = Node(data)
90         if not self.head:
91             self.head = new_node
92             return
93         last_node = self.head
94         while last_node.next:
95             last_node = last_node.next
96         last_node.next = new_node
97     def traverse(self):
98         """Traverse the list and return a list of node data."""
99         elements = []
100         current_node = self.head
101         while current_node:
102             elements.append(current_node.data)
103             current_node = current_node.next
104         return elements
105 # Example usage:
106 if __name__ == "__main__":
107
108     linked_list = SinglyLinkedList()
109     linked_list.insert(1)
110     linked_list.insert(2)
111     linked_list.insert(3)
112     print("Linked List Elements:", linked_list.traverse()) # Output: Linked List Elements: [1, 2, 3]
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

▼ TERMINAL

```
● PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "C:/Users/Ramya Sri/OneDrive/New folder/AI Assistant coding/.
● (.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant
ions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56484' '--' 'c:\Users\Ramya Sri\OneDrive\New folder
Linked List Elements: [1, 2, 3]
```

TASK 4:

Binary Search Tree Operations

PROMPT:

Implement a Binary Search Tree with AI support focusing on insertion and traversal.

CODE & OUTPUT:

```

115 #Implement a Binary Search Tree with AI support focusing on insertion and traversal.
116 class TreeNode:
117     def __init__(self, key):
118         self.left = None
119         self.right = None
120         self.val = key
121 class BinarySearchTree:
122     def __init__(self):
123         self.root = None
124     def insert(self, key):
125         """Insert a new node with the given key into the BST."""
126         if self.root is None:
127             self.root = TreeNode(key)
128         else:
129             self._insert_recursively(self.root, key)
130     def _insert_recursively(self, node, key):
131         if key < node.val:
132             if node.left is None:
133                 node.left = TreeNode(key)
134             else:
135                 self._insert_recursively(node.left, key)
136         else:
137             if node.right is None:
138                 node.right = TreeNode(key)
139             else:
140                 self._insert_recursively(node.right, key)
141     def inorder_traversal(self):
142         """Perform in-order traversal and return a list of keys."""
143         return self._inorder_recursively(self.root)
144     def _inorder_recursively(self, node):
145         res = []
146         if node:
147             res = self._inorder_recursively(node.left)
148             res.append(node.val)
149             res = res + self._inorder_recursively(node.right)
150         return res
151 # Example usage:
152 if __name__ == "__main__":
153     bst = BinarySearchTree()
154     bst.insert(5)
155     bst.insert(3)
156     bst.insert(7)
157     bst.insert(2)
158     bst.insert(4)
159     bst.insert(6)
160     bst.insert(8)
161     print("In-order Traversal:", bst.inorder_traversal()) # Output: In-order Traversal: [2, 3, 4, 5, 6, 7, 8]

```

```

● PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "C:/Users/Ramya Sri/OneDrive/New folder/AI Assistant coding/.venv" PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55649' '--' 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding'
● In-order Traversal: [2, 3, 4, 5, 6, 7, 8]
● (.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\.venv" PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55649' '--' 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding'

```

TASK 5:

Hash Table Implementation

PROMPT:

Create a hash table using AI with collision handling

CODE & OUTPUT:

```

164
165 #Create a hash table using AI with collision handling
166 class HashTable:
167     def __init__(self, size=10):
168         self.size = size
169         self.table = [[] for _ in range(size)] # Using chaining for collision handling
170     def _hash(self, key):
171         """Generate a hash for the given key."""
172         return hash(key) % self.size
173     def insert(self, key, value):
174         """Insert a key-value pair into the hash table."""
175         index = self._hash(key)
176         # Check if the key already exists and update it
177         for i, (k, v) in enumerate(self.table[index]):
178             if k == key:
179                 self.table[index][i] = (key, value) # Update existing key
180                 return
181         # If key does not exist, add new key-value pair
182         self.table[index].append((key, value))
183     def get(self, key):
184         """Retrieve the value associated with the given key."""
185         index = self._hash(key)
186         for k, v in self.table[index]:
187             if k == key:
188                 return v
189         raise KeyError(f"Key '{key}' not found in hash table.")
190     def delete(self, key):
191         """Delete the key-value pair associated with the given key."""
192         index = self._hash(key)
193         for i, (k, v) in enumerate(self.table[index]):
194             if k == key:
195                 del self.table[index][i]
196                 return
197         raise KeyError(f"Key '{key}' not found in hash table.")
198 # Example usage:
199 if __name__ == "__main__":
200     hash_table = HashTable()
201     hash_table.insert("name", "Alice")
202     hash_table.insert("age", 30)
203     print("Name:", hash_table.get("name")) # Output: Name: Alice
204     print("Age:", hash_table.get("age")) # Output: Age: 30
205     hash_table.delete("name")
206     # print(hash_table.get("name")) # Uncommenting this line will raise KeyError: Key 'name' not found in hash table.
207

```

```

● PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\.venv\Scripts\Activate.ps1"
(.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & 'c:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\.venv\Scripts\python.exe' -i -c 'from hash_table import HashTable; ht = HashTable(); ht.insert("name", "Alice"); ht.insert("age", 30); print("Name:", ht.get("name")); print("Age:", ht.get("age")); ht.delete("name"); # print(ht.get("name"))'
● Name: Alice
Age: 30
(.venv) PS C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding> & "C:\Users\Ramya Sri\OneDrive\New folder\AI Assistant coding\.venv\Scripts\python.exe" -i -c 'from hash_table import HashTable; ht = HashTable(); ht.insert("name", "Alice"); ht.insert("age", 30); print("Name:", ht.get("name")); print("Age:", ht.get("age")); ht.delete("name"); # print(ht.get("name"))'

```